C Hackathon Design Notes

GROUP C11

Location: Kolkata

Team members

Aaditya Kumar Choudhary (ACHOUDH4)

Jitendra Singh (JSINGH15)

Sneha Lahiri (SLAHIRI)

Snigdhendu Chattopadhyay (SCHATTOP)

Tamanna Parween (TPARWEEN)

Contents

Scheduler Flow

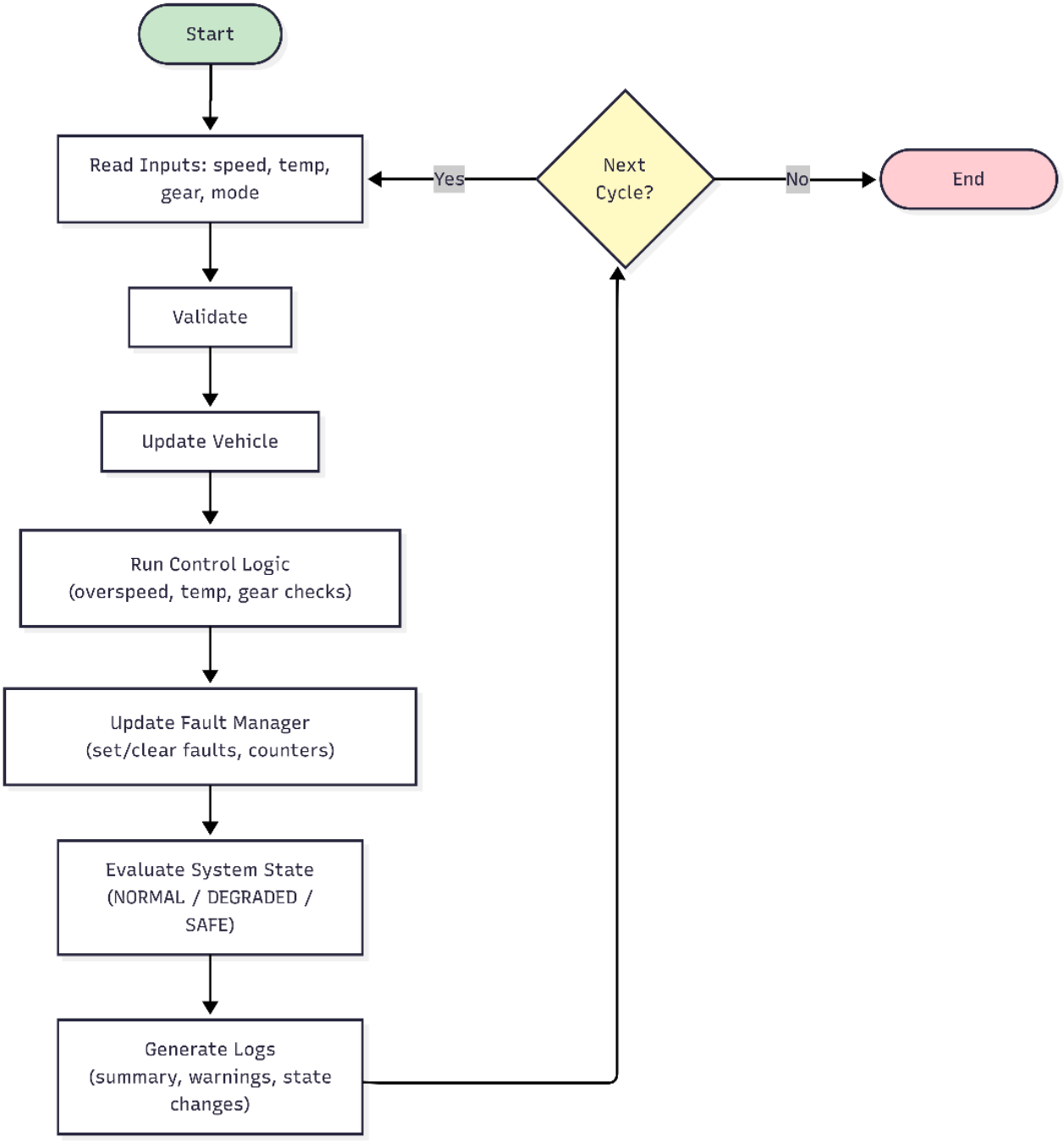


Fig 1: Scheduler Workflow

The scheduler is the heart of the ECU application. It is a continuous loop (while(1)) that runs the same set of tasks in a fixed, strict order every cycle, just like a real embedded ECU.

This is called a cyclic execution model.

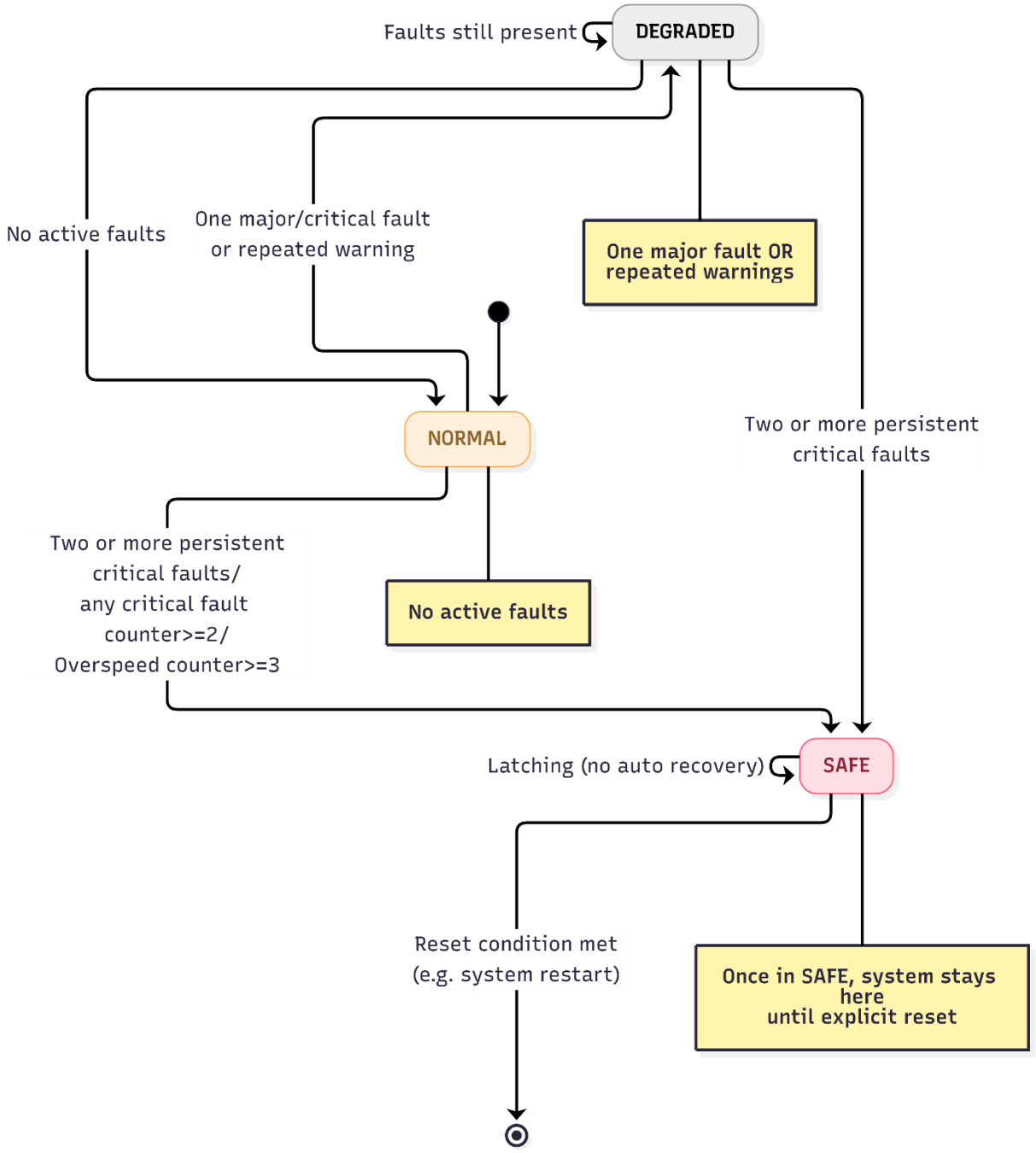
Scheduler Flow per Cycle

Step	Function Called	What it does	Why it is placed here
1	read_inputs() + validate_inputs()	Reads raw values (speed, temp, gear, requested mode) and validates them. Uses last valid value if invalid.	Must be first — everything else needs clean, validated data
2	update_mode()	Checks if mode transition (OFF → ACC → IGNITION_ON etc.) is legal. Forces FAULT mode on illegal transition.	Needs validated input from Step 1. Mode affects control decisions
3	run_control_checks()	Evaluates overspeed (>120), temperature thresholds (>110 critical, >95 high), and gear validity	Needs current mode + validated inputs from previous steps
4	update_fault_status()	Sets bitwise fault flags, increments counters for each fault type	Needs the results of control checks and mode update
5	evaluate_system_state()	Decides NORMAL / DEGRADED / SAFE based on active faults + counters	Needs the latest fault flags and counters from Step 4
6	log_cycle_summary()	Prints full cycle summary + warnings + faults + state changes (to console + log.txt)	Must be last so it shows the complete picture of the cycle
7	Loop back (or wait for next input)	Starts the next cycle immediately (in test mode) or waits for user input	Keeps the cyclic behavior

Module Responsibilities

Name	File	Primary Responsibility	Data Consumed
Input	input.c	Sanitization: Clamps out-of-bounds sensor data and replaces garbage with "Last Valid" values.	VehicleInput, VehicleStatus
Mode	mode.c	State Machine: Validates if the mode request (e.g., OFF to ACC) is legal based on transition rules.	VehicleStatus, VehicleInput
Control	control.c	Safety Cross-Checks: The logic engine that compares different inputs (e.g., "Is speed too high for this gear?").	VehicleInput, VehicleStatus
Fault	fault.c	Diagnostic Manager: Handles the bitwise math to set/clear faults and increments persistent fault counters.	FaultStatus
State	state.c	The Arbiter: Evaluates the severity of all faults to decide the final health state (NORMAL, SAFE).	VehicleStatus, FaultStatus
Log	log.c	Translation: Converts the binary data from all structures into human-readable text and CSV rows.	Input, Status, Faults

State Transition Logic



Fault Handling

The fault handling strategy in this project is designed around Functional Safety principles (similar to ISO 26262), ensuring that the ECU remains predictable and safe even when sensors or software transitions fail.

The strategy follows a Three-Phase Architecture:

1. Detection (The "Flags" Phase)

Faults are detected locally in specialized modules (primarily control.c and mode.c).

- **Decoupling:** The detection logic does *not* decide how the vehicle should react; it only identifies the problem.
- **Bitwise Storage:** All active faults are stored in a single uint32_t bitmask. This allows the system to track multiple simultaneous faults (e.g., Overspeed AND Invalid Gear) using zero extra memory.

2. Management (The "Persistence" Phase)

Located in **lib/fault.c**, this layer filters "noise" from true hardware failure.

- **Fault Counters**: Every fault type has a dedicated counter.
- **Persistence Logic:** The system distinguishes between a Transient Fault (a one-cycle glitch that disappears) and a Persistent Fault (a failure that stays active).
    - *Example*: An overspeed condition is only considered "critical" after it has persisted for 3 consecutive cycles.

3. Arbitration (The "Safe-State" Phase)

Located in **lib/state.c**, this is the final decision-maker.

- **Escalation Hierarchy:** The system evaluates the bitmask and the counters to decide the system health:
    - **NORMAL:** Zero active faults.
    - **DEGRADED:** One major/critical fault or persistent warnings (e.g., High Temp warning). The vehicle remains operational but restricted.
    - **SAFE:** Critical risk detected (Two or more persistent critical faults/any critical fault counter>=2/Overspeed counter>=3).
- **Latching:** The SAFE state is latched. Even if the temperature drops back to normal, the ECU stays in SAFE mode to protect the hardware until a human-initiated reset (MODE_OFF) occurs.

Key Strategy Highlights:

- **Deterministic Priorities:** Faults are ranked from Priority 1 (Critical Heat) to Priority 4 (High Temp Warning). If multiple faults occur, the logging and reporting system always prioritizes the most dangerous one first.
- **Fail-Safe by Default:** If an unknown or "garbage" input is detected, the input.c module automatically reverts to the "Last Valid Value," ensuring the internal logic never operates on corrupted data.

Reset Logic implementation

The Key-Cycle Reset is a standard automotive safety pattern that ensures persistent faults are cleared only under safe conditions.

Step-by-step flow:

1. Reset Request Received The ECU receives a reset request (via "reset": true). It does not reset immediately and instead enters “Pending Reset” state.
2. Safety Interlock Check If the current mode is IGNITION_ON or ACC, the reset is rejected. Warning logged: “Safety Lock: System must be in OFF mode to perform a full reset.”
3. Graceful Shutdown The user must perform legal mode transitions: IGNITION_ON → ACC → OFF.
4. Mode Verification Once the mode becomes MODE_OFF, the pending reset is accepted.
5. Persistent Counter Clearance
    - All fault counters are reset to 0.
    - System state is restored to NORMAL.
    - Active fault flags are cleared.
6. Power-On Self-Test On the next cycle, the ECU starts fresh in NORMAL state, allowing normal mode transitions again.

Future Enhancement:

1. Real-Time Multithreading:

The software should be split into independent "lanes" called threads. A **High-Priority Thread** handles safety-critical checks (like faults), while a **Low-Priority Thread** handles logging. This ensures that a slow task, like writing to log.txt, never delays the ECU from reacting to an emergency.

2. Deterministic Logic (Lookup Tables)

Replace long lists of if/else statements with **Lookup Tables**. By pre-mapping every possible mode change in a simple grid, the software makes decisions instantly. This removes "timing spikes" and ensures the ECU responds in the exact same amount of time every cycle.

3.Time triggered Architecture Design:

Group tasks into fixed-period tasks, which ensures determinism-knowing exactly when a fault will be caught regardless of CPU load, which is a requirement for safety-critical systems.

4.Stack Overflow

Add a module that monitors memory integrity during the execution, ensuring no stack overflows occur during high-frequency processing.

5.Thermal & Load Modeling

Implement a feedback loop where Speed and Gear affect the Temperature over time, making test cases more dynamic and physically grounded.

6.Integrated Watchdog Timer (WDT)

Add a simulated hardware watchdog that must be "kicked" by the main scheduler. If any module (like validate_inputs) hangs, the WDT forces a hard reset.

7. Diagnostic "Freeze Frames"

We have fault bits and counters. When a fault counter first hits the threshold (e.g., moves from 2 to 3 for Overspeed), capture a "Freeze Frame.". Save a snapshot of *all* inputs (Speed, Temp, Gear, Mode) at that exact microsecond. This is how real mechanics diagnose why a car failed—they look at the conditions during the "Fault Trigger" event.